

PROJECT ASSIGNMENT

Design and Implementation of an Optimal Binary Search Tree using Dynamic Programming

Design and Analysis of Algorithms

1. Project Overview

The objective of this project is to design and implement an Optimal Binary Search Tree (OBST) using Dynamic Programming. The project will require you to formulate the problem as a set of overlapping subproblems, construct the Dynamic Programming tables, determine the minimum expected search cost, and reconstruct the resulting optimal binary search tree.

The project should demonstrate both your understanding of the Dynamic Programming approach and your ability to implement, test, and analyze an algorithm.

2. Problem Description

Given a set of sorted keys and their corresponding successful-search probabilities, construct a Binary Search Tree that minimizes the expected number of comparisons required to search for a key.

Keys with higher search probabilities should generally be placed closer to the root because this can reduce the expected search cost.

3. Project Requirements

- Explain the Optimal Binary Search Tree problem and why the arrangement of keys affects search cost.
- Formulate the problem using Dynamic Programming.
- Clearly define the DP state $C[i,j]$, representing the minimum expected search cost for keys i through j .
- Derive and explain the recurrence relation used by the algorithm.
- Define and implement the base cases.
- Construct the DP cost table.
- Construct a root table that records the selected root for each subproblem.
- Reconstruct and display the actual Optimal Binary Search Tree from the root table.
- Calculate and report the minimum expected search cost.
- Verify the resulting expected cost using the constructed tree.
- Analyze the time and space complexity of the implemented algorithm.
- Perform experiments using multiple input sizes and discuss the results.

4. Dynamic Programming Formulation

Let the sorted keys be:

$$K = \{k_1, k_2, \dots, k_n\}$$

and let p_i be the successful-search probability associated with key k_i .

Define:

$C[i,j]$ = minimum expected search cost for keys k_i through k_j

For a single key:

$$C[i,i] = p_i$$

For an interval containing multiple keys, consider every possible root k as follows:

$$C[i,j] = \min \{C[i,k-1] + C[k+1,j]\} + \sum p_s \text{ for } 1 \leq i \leq j \leq n$$

$$C[i,i] = p_i \text{ for } 1 \leq i \leq n$$

The summation term is added because when a subtree is placed one level deeper, every key in that subtree requires one additional comparison.

5. Input Requirements

Your program should accept:

- The number of keys n .
- n sorted keys.
- The successful-search probability for each key.
- The probabilities should be non-negative and should sum to 1 (allowing for minor floating-point rounding).

6. Required Output

Your program must display or report:

- The input keys and their probabilities.
- The Dynamic Programming cost table.
- The root table.
- The minimum expected search cost.
- The structure of the Optimal Binary Search Tree.
- The level/depth of each key in the resulting tree.
- The expected search cost calculated from the final tree.
- The execution time for the tested input sizes.

7. Example Input

Key	Search Probability
10	0.10
20	0.20

30	0.40
40	0.20
50	0.10

The above input is provided as an example. You should also test your implementation with additional inputs of different sizes.

8. Comparison with a Conventional BST

For at least one input set, construct a conventional BST using the same keys and compare it with the Optimal BST.

- Expected search cost.
- Average search depth.
- Tree structure.
- Construction/execution time where meaningful.

9. Experimental Analysis

Test your implementation with several values of n . Recommended sizes are:

$n = 5, 10, 20, 50, 100$

For each input size, record the execution time and discuss how the running time changes as n increases.

Present your experimental results in a table such as:

Number of Keys (n)	Execution Time	Minimum Expected Cost	Remarks

10. Complexity Analysis

Explain the theoretical complexity of your algorithm and relate it to your experimental observations.

Standard OBST Dynamic Programming: Time = $O(n^3)$, Space = $O(n^2)$

Your report should explain why the standard algorithm has cubic time complexity and quadratic space complexity.

11. Implementation Guidelines

- Use a programming any language you comfortable
- Write modular and readable code.
- Use meaningful variable and function names.
- Include comments for important parts of the implementation.
- Do not hard-code the final tree or the minimum cost.

- The program must compute the result from the supplied input.
- Handle invalid inputs appropriately, such as probabilities that are negative or do not sum approximately to 1.

12. Report Structure

Submit a report containing the following sections:

1. Introduction
2. Problem Definition
3. Dynamic Programming Formulation
4. Algorithm / Pseudocode
5. Implementation
6. Experimental Results
7. Comparison with a Conventional BST
8. Complexity Analysis
9. Discussion
10. Conclusion
11. References

13. Submission

Submit the following:

- Project report in PDF format.
- Complete source code.
- A short README file explaining how to compile/run the program and how to provide input.
- Sample input and output.

Important: The implementation should be your own work. You should be able to explain your algorithm, code, DP tables, and experimental results during the project evaluation.

14. Evaluation Criteria

Component	Weight	What is Evaluated
Problem Understanding	10%	Clear explanation of the OBST problem and objective.
DP Formulation	20%	State definition, recurrence, base cases, and explanation.
Implementation	20%	Correct and well-structured implementation.
Tree Reconstruction	15%	Correct use of the root table to construct the final BST.
Experimental Analysis	15%	Testing with multiple input sizes and discussion of results.
Complexity Analysis	10%	Correct theoretical time and space analysis.
Presentation	10%	Clarity, organization, and ability to explain the work.

